Day 131 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 6 reference

Watch: Contrastive Learning with SimCLR | Deep Learning Animated (Deepia) | Contrastive Learning of visual representations - SimCLR (paper illustrated) (AI Bites)

`day131_contrastive_learning.py`, segment by segment, in the order `main()` uses things. The views (augment), the loss and its gradient (nt_xent), training (train_contrastive) and the representation (represent) are new; the network, its backward pass and Adam are Day 126's, imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = images, one per row · V = views · Z = projections · U = unit vectors · S = similarities over τ · P = softmax probabilities · pos = each row's partner

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
for w in ("week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day123_exercise as d123
    import day119_exercise as d119
    import day116_exercise as d116
SIZES = [784, 256, 128, 64]
TAU = 0.1
```

- `with contextlib.redirect_stdout(io.StringIO()):` — earlier lessons print their own checks when imported; this sends that output nowhere.
- `SIZES = [784, 256, 128, 64]` — the encoder ends at 128 numbers (the representation); the head maps them to 64.
- `TAU = 0.1` — the temperature, chosen in exploration and examined in section 6.

Segment 1 — The views and the loss

augment: a random view of every image at once

MATH

$$source(r, c) = R(-a)\,(r, c) - (dy, dx) \quad \text{(nearest pixel)}$$

CODE

```
def augment(X, rng, shift=3, angle=15, erase=8, noise=0.1):
    n = len(X); a = np.radians(rng.uniform(-angle, angle, n)); dx, dy = rng.randint(-shift, shift
        + 1, (2, n))
    r, c = np.mgrid[0:28, 0:28].reshape(2, -1) - 13.5
    cos, sin = np.cos(a)[:, None], np.sin(a)[:, None]
    src_r = np.rint(cos * r + sin * c + 13.5 - dy[:, None]).astype(int)
    src_c = np.rint(-sin * r + cos * c + 13.5 - dx[:, None]).astype(int)
    ok = (src_r >= 0) & (src_r < 28) & (src_c >= 0) & (src_c < 28)
    V = np.where(ok, X[np.arange(n)[:, None], np.clip(src_r, 0, 27) * 28 + np.clip(src_c, 0, 27)],
        0).astype(np.float32)
    if erase:
        r0, c0 = rng.randint(0, 28 - erase, (2, n)); hit = rng.uniform(size=n) < 0.5
        V = V.reshape(n, 28, 28)
        for i in np.where(hit)[0]:
            V[i, r0[i]:r0[i] + erase, c0[i]:c0[i] + erase] = 0
        V = V.reshape(n, -1)
    return np.clip(V + rng.normal(0, noise, V.shape).astype(np.float32), 0, 1)
```

- `np.mgrid[0:28, 0:28].reshape(2, -1) - 13.5` — every pixel's row and column, measured from the image centre.
- `cos * r + sin * c + 13.5 - dy[:, None]` — for each output pixel, where it came from: rotated back and shifted back; [:, None] gives one row per image.
- `np.rint(...).astype(int)` — round to the nearest pixel (no interpolation).
- `X[np.arange(n)[:, None], ...]` — fancy indexing: image i reads its own source pixels.
- `np.where(ok, ..., 0)` — pixels whose source falls outside the image become black.
- `V[i, r0[i]:r0[i] + erase, c0[i]:c0[i] + erase] = 0` — erase a square patch in the chosen images.
- `np.clip(V + noise, 0, 1)` — noise added, then pixels kept in [0, 1].

nt_xent: the loss and its hand-derived gradient

MATH

$$L_i = -\log \operatorname{softmax}_{j \neq i}(u_i \cdot u_j / \tau)_{p(i)}$$

$$\partial L/\partial z = (g - u\,(u \cdot g)) / |z|, \quad g = \partial L/\partial u$$

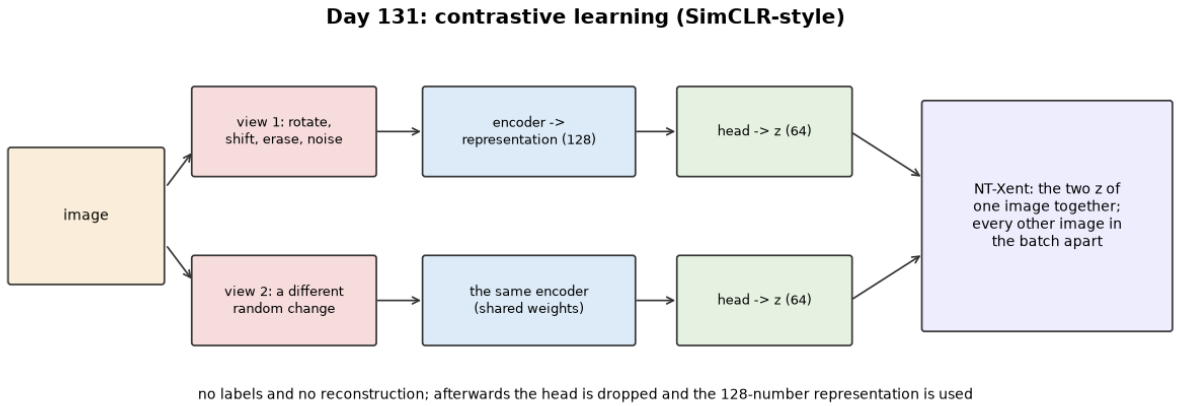
CODE

```
def nt_xent(Z, tau):
    n2 = len(Z); N = n2 // 2; norm = np.linalg.norm(Z, axis=1, keepdims=True) + 1e-12; U = Z /
        norm
    S = U @ U.T / tau; np.fill_diagonal(S, -np.inf)
    pos = np.r_[np.arange(N, n2), np.arange(N)]
    m = S.max(1, keepdims=True); E = np.exp(S - m); P = E / E.sum(1, keepdims=True)
    loss = float(np.mean(-np.log(P[np.arange(n2), pos] + 1e-30)))
    dS = P.copy(); dS[np.arange(n2), pos] -= 1; dS /= n2
    dU = (dS + dS.T) @ U / tau
    dZ = (dU - U * (U * dU).sum(1, keepdims=True)) / norm
    return loss, dZ, P[np.arange(n2), pos]
```

- `U = Z / norm` — unit vectors, so U @ U.T holds cosines.

- `np.fill_diagonal(S, -np.inf)` — $e^{-\infty} = 0$: a view is never its own candidate.
- `pos = np.r_[np.arange(N, n2), np.arange(N)]` — row i's partner is i + N, and row i + N's is i.
- `dS[np.arange(n2), pos] -= 1` — the softmax gradient: probability minus a one at the right answer.
- `(dS + dS.T) @ U / tau` — each u appears in a row and a column of S, so both directions contribute.
- `dU - U * (U * dU).sum(1, keepdims=True)` — remove the part along u: lengthening u changes nothing after normalising.

SYSTEM DIAGRAM



Two views, one shared network, one loss.

Segment 2 — Training and judging

train_contrastive and represent

CODE

```
def train_contrastive(X, sizes, tau=0.5, epochs=20, batch=256, lr=1e-3, seed=0, aug=None):
    acts = ["relu"] * (len(sizes) - 2) + ["linear"]; net = d126.init_net(sizes, seed); opt =
        d126.Adam(net, lr)
    rng = np.random.RandomState(seed + 1); aug = aug or augment; hist = []
    for _ in range(epochs):
        o = rng.permutation(len(X)); losses = []
        for s in range(0, len(X) - batch + 1, batch):
            xb = X[o[s:s + batch]]; V = np.vstack([aug(xb, rng), aug(xb, rng)])
            A = d126.forward(net, V, acts); loss, dZ, _ = nt_xent(A[-1], tau)
            opt.step(net, d126.backward(net, A, acts, dZ)[0]); losses.append(loss)
        hist.append(float(np.mean(losses)))
    return net, hist
def represent(net, X, layer=-2):
    acts = ["relu"] * (len(net) - 1) + ["linear"]
    return d126.forward(net, X, acts)[layer]
```

- `aug = aug or augment` — use the caller's view function if given (section 7), otherwise the full one.
- `range(0, len(X) - batch + 1, batch)` — full batches only, so every step has the same 510 negatives per view.
- `np.vstack([aug(xb, rng), aug(xb, rng)])` — rows 0..255 are first views, 256..511 second views: the layout nt_xent expects.
- `d126.backward(net, A, acts, dZ)[0]` — Day 126's backward pass, fed the loss's gradient; [0] keeps the weight gradients.
- `[layer] with layer=-2` — the activations one layer before the output: the 128-number representation.

probe: a linear classifier with few labels

CODE

```
def probe(R, y, n_per, seed=0):
    rng = np.random.RandomState(seed)
    tr = np.concatenate([rng.choice(np.where(y[:60000] == c)[0], n_per, replace=False) for c in
        range(10)]); te = np.arange(60000, 70000)
    mu, sd = R[tr].mean(0), R[tr].std(0) + 1e-6
    W, b = d116.fit_softmax(((R[tr] - mu) / sd).astype(np.float64), y[tr], 10, lam=1e-2,
        steps=500)
    return float(np.mean((((R[te] - mu) / sd) @ W + b).argmax(1) == y[te]))
```

- `rng.choice(np.where(y[:60000] == c)[0], n_per, replace=False)` — n_per training images of digit c, without repeats.
- `mu, sd = R[tr].mean(0), R[tr].std(0) + 1e-6` — standardise with the labelled images' statistics only.
- `d116.fit_softmax(..., lam=1e-2, steps=500)` — Day 116's softmax regression, unchanged.

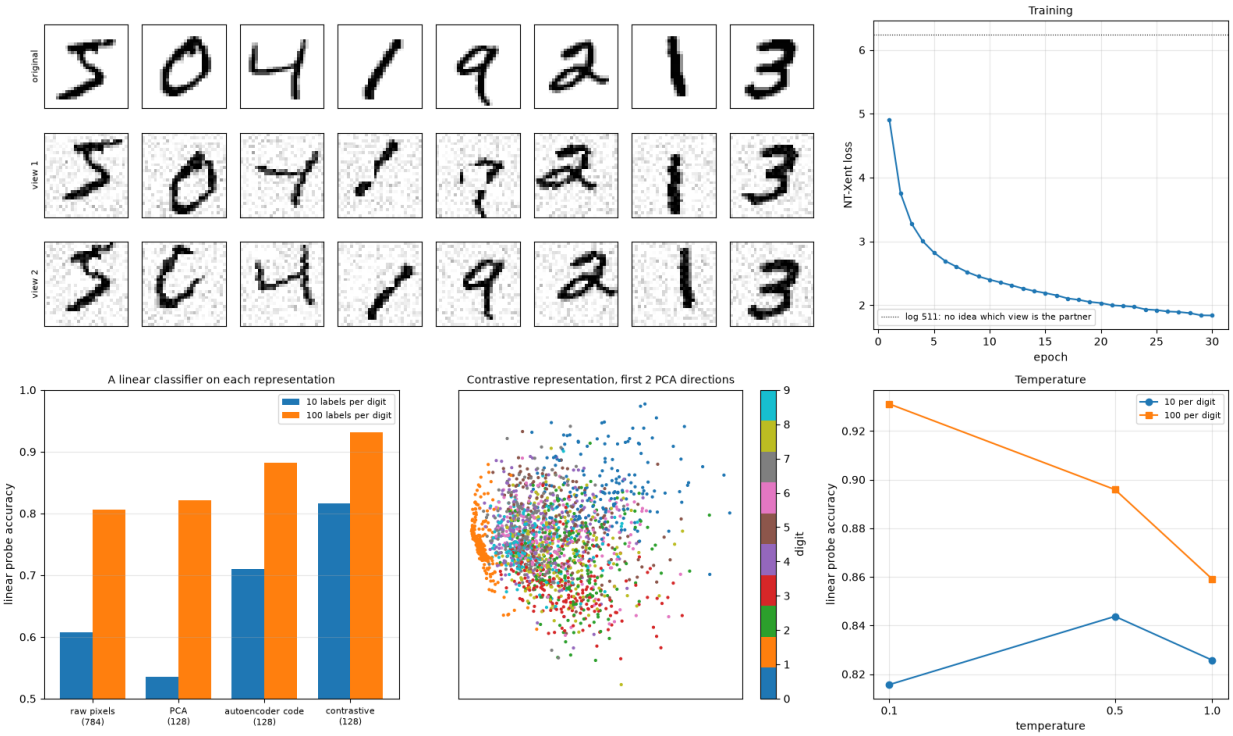
demo_views_matter: training with some changes switched off

CODE

```
variants = {"noise": dict(shift=0, angle=0, erase=0), "noise + shift": dict(angle=0, erase=0),
            "noise + shift + rotation": dict(erase=0)}
rows = []
for name, kw in variants.items():
    net, _ = train_contrastive(X[:20000], SIZES, TAU, epochs=30, aug=lambda Xb, rng, kw=kw:
        augment(Xb, rng, **kw)); R = represent(net, X)
    rows.append((name, np.mean([probe(R, y, 10, s) for s in range(3)]), probe(R, y, 100)))
```

- `dict(shift=0, angle=0, erase=0)` — keyword arguments stored in a dict; the noise keeps its default 0.1.
- `lambda Xb, rng, kw=kw: augment(Xb, rng, **kw)` — kw=kw freezes this variant's settings inside the lambda; ** unpacks them as keywords.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.mgrid[0:28, 0:28]` — two grids of row and column numbers.
- `np.fill_diagonal(S, v)` — set the diagonal in place.
- `np.r_[a, b]` — join ranges into one array.
- `A[np.arange(n), idx]` — pick one entry per row.
- `f(**kw)` — pass a dict as keyword arguments.
- `lambda x, kw=kw: ...` — freeze a loop variable inside a lambda.

End of Day 131 syntax reference. For the experiments and what they showed, see the Day 131 complete guide.